

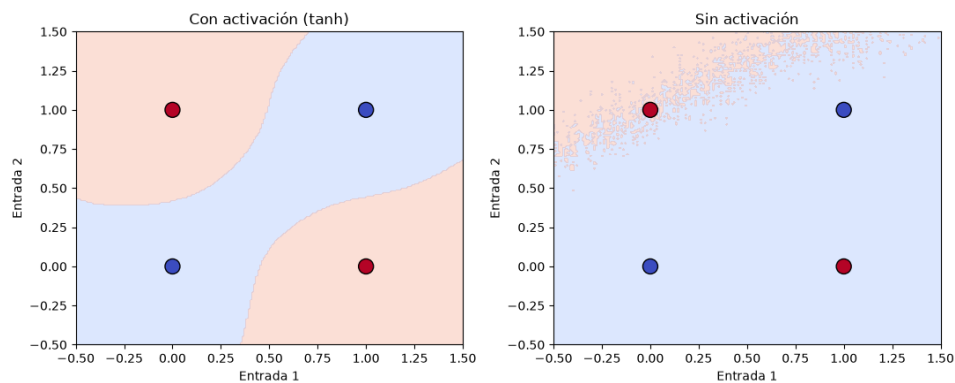
## Laboratorio 7

### Sección 1. Redes multicapa con nn.Module

#### **Resultado del MLP en XOR y frontera de decisión**

Para resolver XOR se armó una clase MLP(nn.Module) con una capa oculta de 8 neuronas y activación tanh, entrenada durante 2000 épocas con Adam. El modelo llegó a 100% de exactitud y a una pérdida prácticamente en 0. Las probabilidades de salida fueron casi 0 para las entradas [0,0] y [1,1] y casi 1 para [0,1] y [1,0].

La segunda versión es idéntica pero sin la función de activación. Terminó con una pérdida de 0.6931 y 75% de exactitud, y le dio una probabilidad de más o menos 0.5 a las cuatro entradas, así que no aprendió el patrón. Ese 75% es lo máximo que puede lograr un modelo lineal en XOR, no es que haya aprendido algo. En la Figura 1 se ve que con tanh la frontera es no lineal y deja los cuatro puntos del lado correcto, mientras que sin activación no hay forma de separarlos.



#### **¿Por qué apilar capas lineales equivale a una sola capa lineal?**

Si entre dos capas lineales no hay una activación, la salida de la primera entra directo a la segunda y al combinar las dos operaciones el resultado sigue siendo una operación lineal. Esto se comprobó con la red sin activación. La primera capa tenía pesos de forma (8, 2) y la segunda de forma (1, 8), y al multiplicarlos sale una sola matriz equivalente de forma (1, 2). Las salidas con las dos capas y con la capa equivalente fueron prácticamente iguales y torch.allclose() devolvió True. Por eso, sin una activación no lineal la red se comporta como una sola capa lineal y no puede resolver XOR.

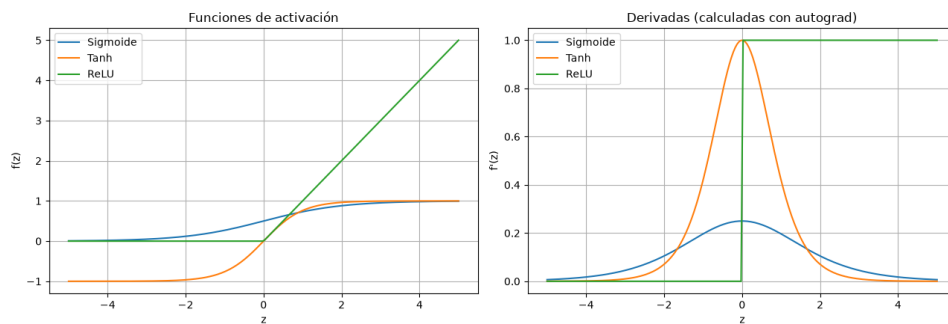
## Parámetros y formas de la red de la diapositiva

La red de la diapositiva tiene una entrada de 3, dos capas ocultas de 4 y una salida de 2, y en total suma 46 parámetros. La Tabla 1 muestra la forma de cada tensor de pesos y de sesgos.

| Parámetro            | Forma  | Cantidad |
|----------------------|--------|----------|
| capa_oculta_1.weight | (4, 3) | 12       |
| capa_oculta_1.bias   | (4,)   | 4        |
| capa_oculta_2.weight | (4, 4) | 16       |
| capa_oculta_2.bias   | (4,)   | 4        |
| capa_salida.weight   | (2, 4) | 8        |
| capa_salida.bias     | (2,)   | 2        |

## Sección 2. Funciones de activación

Se calcularon sigmoide, tanh y ReLU sobre 200 valores de  $z$  entre menos 5 y 5, y sus derivadas se obtuvieron con autograd. La Figura 2 muestra las funciones y sus derivadas en dos paneles.



### ¿En qué regiones se desvanece el gradiente de la sigmoide y de tanh?

Tomando como referencia una derivada menor a 0.05, la sigmoide (que tiene una derivada máxima de solo 0.25) se desvanece cuando  $z$  es menor a menos 2.89 o mayor a 2.89. En tanh (derivada máxima casi 1) pasa cuando  $z$  es menor a menos 2.19 o mayor a 2.19. O sea que el problema aparece en los extremos, donde la derivada es muy chica y en el backpropagation los pesos casi no se actualizan.

## ¿Por qué ReLU es la opción por defecto en capas ocultas?

Para valores positivos la derivada de ReLU se queda en 1, así que no se achica cuando  $z$  crece y evita la saturación que tienen sigmoide y tanh. Para valores negativos devuelve 0 y su derivada también es 0, pero en la parte positiva deja pasar el gradiente de forma directa. Además es muy simple de calcular.

## Sección 3. Pérdida, optimizador y ciclo de entrenamiento

### Datos, modelo y ciclo de entrenamiento

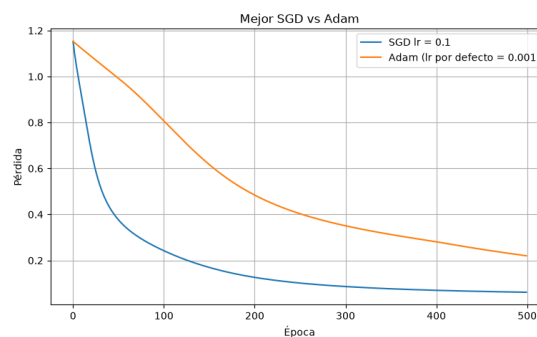
Iris se dividió en 80% para entrenamiento y 20% para validación de forma estratificada (120 y 30 flores) y se estandarizó con la media y la desviación del entrenamiento. El MLP tiene una entrada de 4, una capa oculta de 8 neuronas con ReLU y una salida de 3, y suma 67 parámetros (40 en la primera capa y 27 en la segunda). El ciclo de 500 épocas tiene el forward, la pérdida con `nn.CrossEntropyLoss`, `optimizer.zero_grad()`, `loss.backward()` y `optimizer.step()`.

### ¿Por qué no se pone softmax en la última capa?

`nn.CrossEntropyLoss` ya aplica un log softmax por dentro, por eso espera recibir los logits sin procesar. Si la red también aplicara softmax se haría dos veces, los valores quedarían aplastados entre 0 y 1, los gradientes se debilitan y además es menos estable numéricamente. Por eso la última capa solo devuelve los tres logits.

### Comparación de las tres tasas de aprendizaje

Con  $lr=0.001$  la pérdida pasó de 1.1529 a 1.0153 en 500 épocas, o sea que sí aprende pero demasiado lento (70% en validación). Con  $lr=0.1$  la pérdida bajó a 0.0615 y llegó a 96.67%, así que esa fue la que convergió. Con  $lr=10$  la pérdida se dispara a valores gigantes en las primeras épocas (del orden de 10 elevado a 24 en la gráfica), después baja y termina en 0.7824 con solo 70% de exactitud. Los pasos son tan grandes que SGD pasa de largo cualquier buena solución, por eso es la que diverge.

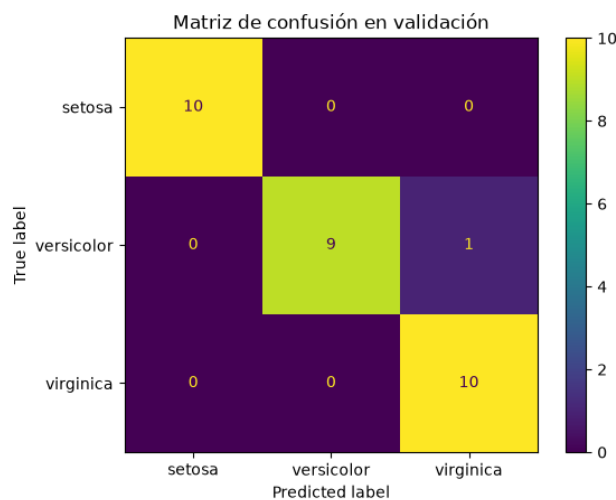


## Adam vs el mejor SGD

La mejor tasa de SGD fue 0.1. Con Adam y su tasa por defecto (0.001) la pérdida final fue 0.2202 contra 0.0615 de SGD, y en validación sacó 90% contra 96.67%. En la Figura 3 (derecha) se ve que Adam baja más despacio y todavía iba bajando al llegar a la época 500. Esto no quiere decir que SGD siempre sea mejor, solo que aquí la tasa de 0.1 funcionó mejor.

## Exactitud en validación y matriz de confusión

El mejor modelo (SGD con  $\text{lr}=0.1$ ) tuvo una exactitud de 96.67% en validación, calculada dentro de `model.eval()` y `torch.no_grad()`. La Tabla 2 muestra la matriz de confusión. El único error fue una flor versicolor clasificada como virginica, así que el par de especies que más confunde es versicolor y virginica. Setosa quedó totalmente separada de las otras dos.



## Sección 4. Redes recurrentes para etiquetado POS

### Datos y modelo

Se usó el treebank de NLTK con el tagset universal, que tiene 3914 oraciones (3131 para entrenamiento y 783 para prueba). El vocabulario quedó en 4844 entradas, porque solo se guardaron las palabras que aparecen al menos dos veces en el entrenamiento (de 10138 distintas), y hay 12 etiquetas más el relleno. El modelo es un `nn.Embedding` de 100 dimensiones, un `nn.LSTM` de 128 neuronas y una capa `nn.Linear` que da una etiqueta por palabra. Los batches se armaron con `pad_sequence`, la pérdida usa `ignore_index` para no contar el relleno y se entrenó 10 épocas con Adam.

## Exactitud del LSTM y de la RNN

Los dos modelos quedaron bastante parecidos, pero el LSTM fue un poco mejor con 93.97% contra 93.51% de la RNN (Tabla 3). El LSTM tiene más parámetros porque tiene compuertas extra para decidir qué información guarda o descarta de la oración. Como fue el mejor, se usó el LSTM en las siguientes pruebas.

Tabla 3. Exactitud por token en prueba, sin contar el relleno.

| Modelo | Parámetros | Exactitud por token |
|--------|------------|---------------------|
| LSTM   | 603837     | 93.97%              |
| RNN    | 515517     | 93.51%              |

## Oraciones ambiguas, ¿el contexto resuelve la ambigüedad?

El contexto ayuda, pero no siempre alcanza. Con run funcionó bien. En "They run every morning before work" y en "He will run the company next year" la etiquetó como VERB, y en "The company had a good run this year" como NOUN. Con book fue distinto. En "She read a good book last night" la etiquetó bien como NOUN, pero en "I want to book a flight to Boston" y "Please book the room for tomorrow" también puso NOUN cuando debía ser verbo, así que parece que aprendió una preferencia fuerte por NOUN para esa palabra. Además, flight, Please y room eran palabras desconocidas para el vocabulario, lo que le quita información al contexto.

## Comparación con el tagger del Laboratorio 6

En el Laboratorio 6 el tagger de spaCy (en\_core\_web\_sm) coincidió con el etiquetado manual en 46 de 47 tokens (97.87%), pero con etiquetas Penn Treebank y solo 5 oraciones de novelas, así que no se compara directo con el LSTM. Para hacerlo parejo se etiquetó todo el conjunto de prueba (20549 tokens) con spaCy y sus etiquetas se pasaron al tagset universal (Tabla 4).

Tabla 4. LSTM contra spaCy en el conjunto de prueba del treebank.

| Tagger               | Todos los tokens | Sin tokens X |
|----------------------|------------------|--------------|
| LSTM (PyTorch)       | 93.97%           | 93.73%       |
| spaCy en_core_web_sm | 91.87%           | 96.66%       |

Con todos los tokens gana el LSTM, pero el 6.62% de los tokens de prueba tiene la etiqueta X, que en el treebank son sobre todo elementos vacíos que los anotadores insertaron en las oraciones. El LSTM los aprendió porque los vio en el entrenamiento y spaCy nunca los vio. Sin esos tokens gana spaCy con 96.66% contra 93.73%.

Atribuyo la diferencia sobre todo al tamaño del corpus, porque el LSTM se entrenó desde cero con solo 3131 oraciones y spaCy viene entrenado con muchísimos más datos. También influyen los embeddings, que en el LSTM se aprendieron desde cero con ese corpus chico, y la arquitectura, ya que spaCy usa un modelo más optimizado para etiquetar. Aun así, llegar a cerca de 94% con un LSTM entrenado solo con este corpus es un buen resultado.

## **Sección 9. Conclusiones, del perceptrón a la atención**

### **¿Por qué fue tan importante el caso XOR?**

El caso XOR fue importante porque en 1969 Minsky y Papert mostraron que el perceptrón de una sola capa no lo podía resolver, ya que los puntos no se separan con una sola línea, y eso hizo que se perdiera bastante interés en las redes neuronales por varios años. En el laboratorio se vio directo, con 100% de exactitud usando tanh y 75% sin activación. Agregar más capas no es suficiente, porque si todas son lineales se combinan en una sola.

### **One hot, TF IDF y embeddings densos**

One hot, bag of words y TF IDF producen vectores dispersos. One hot representa cada palabra con un vector del tamaño del vocabulario con un solo 1 y el resto en 0 (con "company" el vector tuvo 4844 posiciones y solo 1 valor distinto de cero). Bag of words y TF IDF hacen algo parecido con documentos completos (como en los laboratorios anteriores) y solo indican presencia, frecuencia o importancia. Además, en one hot todas las palabras quedan igual de lejos unas de otras, así que el modelo no puede ver que dos palabras se parecen.

### **¿Cómo maneja mi LSTM las palabras desconocidas?**

Mi LSTM maneja las palabras desconocidas con un token especial <UNK>. Se dejaron fuera las palabras que aparecen una sola vez en el entrenamiento para que el modelo aprenda un embedding para ese token. En prueba hubo 2134 tokens desconocidos de 20549 (10.38%), con una exactitud de 97.19% en las palabras conocidas y de solo 66.21% en las desconocidas.

ENLACE AL REPOSITORIO: [HTTPS://GITHUB.COM/LUISPEGR25/NLP](https://github.com/LUISPEGR25/NLP)